

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ЛАБОРАТОРНАЯ РАБОТА №3
по дисциплине «Вычислительная математика»
Тема: Численное интегрирование

Студентка гр. 4342

Киселева А.Д.

Преподаватель

Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы

Используя квадратурные формулы прямоугольников, трапеций и Симпсона, программно вычислить значения заданного интеграла и, применив правило Рунге, найти наименьшее значение n (наибольшее значение шага h), при котором каждая из указанных формул дает приближенное значение интеграла с погрешностью ε , не превышающей заданную.

Теоретический минимум.

Повышения точности численного интегрирования добиваются путем применения составных формул. Для этого при нахождении определенного интеграла отрезок $[a, b]$ разбивают на четное $n = 2m$ число отрезков длины $h = (b - a)/n$ и на каждом из отрезков длины $2h$ применяют соответствующую формулу. Таким образом получают составные формулы прямоугольников, трапеций и Симпсона.

На сетке $x_i = a + ih$, $y = f(x_i)$, $i = 0, 1, 2, \dots, 2m$, составные формулы имеют следующий вид:

- формула прямоугольников

$$\int_a^b f(x) dx = h \sum_{i=0}^{n-1} \left(f\left(x_i + \frac{h}{2}\right) \right) + R_1;$$

- формула трапеций

$$\int_a^b f(x) dx = \frac{h}{2} \sum_{i=0}^{n-1} (f(x_i) + f(x_{i+1})) + R_2;$$

- формула Симпсона

$$\int_a^b f(x) dx = \frac{h}{3} \sum_{i=0}^{m-1} (f(x_{2i}) + 4f(x_{2i+1}) + f(x_{2i+2})) + R_3,$$

где R_1, R_2, R_3 - остаточные члены. При $n \rightarrow \infty$ приближенные значения интегралов для всех трех формул (в предположении отсутствия погрешностей округления) стремятся к точному значению интеграла.

Для практической оценки погрешности квадратурной можно использовать правило Рунге. Для этого проводят вычисления на сетках с шагом h и $h/2$, получают приближенные значения интеграла I_h и $I_{h/2}$ и за окончательные значения интеграла принимают величины:

$I_{h/2} + |I_{h/2} - I_h|/3$ - для формулы прямоугольников;

$I_{h/2} - |I_{h/2} - I_h|/3$ - для формулы трапеций;

$I_{h/2} - |I_{h/2} - I_h|/15$ - для формулы Симпсона.

За погрешность приближенного значения интеграла для формул прямоугольников и трапеций тогда принимают величину $|I_{h/2} - I_h|/3$, а для формулы Симпсона $|I_{h/2} - I_h|/15$.

Выполнение работы.

Вид интеграла в соответствии с вариантом 6: $\int_0^1 \cos x^2 dx$

Созданы отдельные функции для вычисления определённого интеграла по формулам прямоугольников, трапеций и Симпсона. Функция `rectangle()` реализует метод прямоугольников, в котором интеграл приближается суммой площадей прямоугольников, высота которых определяется значением подынтегральной функции в середине каждого частичного отрезка. Функция `trapezoid()` реализует метод трапеций, где на каждом частичном отрезке подынтегральная функция заменяется прямой линией, а площадь под кривой — площадью трапеции. Функция `simpson()` реализует метод Симпсона, в котором на каждом удвоенном отрезке подынтегральная функция заменяется квадратичной параболой, проходящей через три точки, и вычисляется аналитически.

Создана отдельная функция $f(x)$, которая вычисляет значение подынтегральной функции $f(x)=\cosh(x^2)$ в заданной точке.

Реализована функция `runge()`, которая автоматически подбирает шаг интегрирования для достижения требуемой точности. Начальное количество разбиений принимается равным $n=2$. На каждом шаге вычисляются приближения интеграла I_h и $I_{h/2}$. Если полученная оценка погрешности меньше заданной точности ε , процесс завершается. В противном случае количество разбиений удваивается и вычисления повторяются.

Головная программа запрашивает у пользователя значение точности ε , задаёт пределы интегрирования $a=0$, $b=1$ и последовательно вызывает функцию `runge()` для методов прямоугольников, трапеций и Симпсона. Для каждого метода выводятся: количество разбиений, при котором достигнута требуемая точность, уточнённое значение интеграла, оценка погрешности Рунге и количество удвоений шага.

Тестирование.

В таблице 1 представлены результаты тестирования программы при различных значениях заданной точности ε .

Таблица 1 - Тестирование программы

ε	Метод прямоугольников				Метод трапеций				Метод Симпсона			
	n	I	error	N	n	I	error	N	n	I	error	N
0.1	2	1.1042 66465 082	0.0055 20053 557	0	2	1.1052 83070 118	0.0115 48409 631	0	2	1.1048 71574 411	0.0004 11495 707	0
0.01	2	1.1042 66465 082	0.0055 20053 557	0	4	1.1047 74767 600	0.0030 14178 037	1	2	1.1048 71574 411	0.0004 11495 707	0
0.001	8	1.1047 35884 620	0.0003 79980 830	2	8	1.1047 40289 413	0.0007 62164 056	2	2	1.1048 71574 411	0.0004 11495 707	0
0.0001	16	1.1047 37810 185	0.0000 95476 599	3	32	1.1047 37948 601	0.0000 47807 507	4	4	1.1047 40880 766	0.0000 33886 835	1

Обозначения: n — начальное количество разбиений исходного отрезка, I — значение интеграла, $error$ — значение ошибки Рунге, N — количество удвоений.

Вывод.

В результате лабораторной работы были программно реализованы методы прямоугольников, трапеций и Симпсона, оценена их погрешность с помощью формул Рунге, вычислено количество удвоений, требуемое для достижения заданной точности. Проведено тестирование, которое показало, что для численного интегрирования наиболее эффективен метод Симпсона который при минимальном количестве удвоений шага достигает наивысшей точности.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД

```
#include <iostream>
#include <vector>
#include <string>
#include <cmath>
#include <iomanip>

double f(double x){
    return std::cosh(x * x);
}

double rectangle(std::vector<double>& arr_x, double h, int n){
    double sum = 0.0;

    for (int i = 0; i < n; ++i){
        sum += f(arr_x[i] + h/2);
    }

    return h * sum;
}

double trapezoid(std::vector<double>& arr_x, double h, int n){
    double sum = 0.0;

    for (int i = 0; i < n; ++i){
        sum += f(arr_x[i]) + f(arr_x[i+1]);
    }

    return h/2 * sum;
}

double simpson(std::vector<double>& arr_x, double h, int m){
    double sum = 0.0;

    for (int i = 0; i < m; ++i){
        sum += f(arr_x[2*i]) + 4*f(arr_x[2*i+1]) + f(arr_x[2*i+2]);
    }
}
```

```

    }

    return h/3 * sum;
}

std::vector<double> make_x_net(double h, int n, double a){
    std::vector<double> arr_x(n + 1);
    for (int i = 0; i <= n; ++i){
        arr_x[i] = a + i*h;
    }
    return arr_x;
}

void runge(int method, int n, double a, double b, double eps){
    std::vector<std::pair<int, double>> ans;
    double runge_error = 1.0;
    double I = 0.0;
    int cnt_incr_n = 0;
    std::string method_name = "";

    switch(method){
        case 1: {
            method_name = "Rectangle";
            break;
        }
        case 2: {
            method_name = "Trapezoid";
            break;
        }
        case 3: {
            method_name = "Simpson";
            break;
        }
        default: {
            std::cout << "Неизвестный метод\n";
            return;
        }
    }
}

```

```

}

std::cout << "\nМетод " << method_name << ":\n";

while (runge_error >= eps){
    int m = n/2;
    double h = (b - a) / n;

    std::vector<double> arr_x = make_x_net(h, n, a);
    std::vector<double> arr_x_2 = make_x_net(h/2, 2*n+1, a);

    switch(method){
        case 1: {
            double I_rectangle_h = rectangle(arr_x, h, n);
            double I_rectangle_h_2 = rectangle(arr_x_2, h/2,
2*n);

            runge_error = fabs(I_rectangle_h_2 - I_rectangle_h)
/ 3.0;

            I = I_rectangle_h_2 + runge_error;
            break;
        }
        case 2: {
            double I_trapezoid_h = trapezoid(arr_x, h, n);
            double I_trapezoid_h_2 = trapezoid(arr_x_2, h/2,
2*n);

            runge_error = fabs(I_trapezoid_h_2 - I_trapezoid_h)
/ 3.0;

            I = I_trapezoid_h_2 - runge_error;
            break;
        }
        case 3: {
            double I_simpson_h = simpson(arr_x, h, m);
            double I_simpson_h_2 = simpson(arr_x_2, h/2, n);
            runge_error = fabs(I_simpson_h_2 - I_simpson_h) /
15.0;

            I = I_simpson_h_2 - runge_error;
            break;
        }
    }
}

```

```

        }
    }

    std::cout << "n = " << n << ", I = " << I << ", error = "
<< runge_error << '\n';
    n *= 2;
    cnt_incr_n++;
}

std::cout << "Количество удвоений: " << cnt_incr_n-1 << '\n';
}

int main(){
    std::cout << std::fixed << std::setprecision(12);

    double eps;
    std::cout << "Введите значение прогрешности epsilon:\n";
    std::cin >> eps;

    double a = 0.0;
    double b = 1.0;

    runge(1, 2, a, b, eps);
    runge(2, 2, a, b, eps);
    runge(3, 2, a, b, eps);

    return 0;
}

```